



Đồ Án Computer Vision ver 2

Công nghệ thông tin (Trường Đại học Ngoại ngữ - Tin học Thành phố Hồ Chí Minh)



messages.pdf_cover_qr_code_label

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN
&



BÁO CÁO KẾT THÚC HỌC PHẦN
Thị Giác Máy Tính

NHẬN DIỆN CỬ CHỈ TAY THỜI GIAN THỰC

Giảng viên hướng dẫn: **ThS. Nguyễn Thị Mỹ Hạnh**

Sinh viên thực hiện:

- | | |
|-------------------------|------------|
| 1. Danh Hoàng Hiếu Nghị | 23DH112270 |
| 2. Lê Công Hải Nam | 23DH114772 |
| 3. Thi Minh Khoa | 23DH114397 |

Thành phố Hồ Chí Minh, ngày 14 tháng 12 năm 2025

MỤC LỤC

LỜI CẢM ƠN.....	2
NHẬN XÉT CỦA GIẢNG VIÊN	3
CHƯƠNG 1. TỔNG QUAN.....	6
1.1. Đặt vấn đề	6
1.2. Mục tiêu đề tài.....	7
1.3. Lý do chọn đề tài.....	7
1.4. Mục đích của đề tài.....	Error! Bookmark not defined.
1.4.1. Mục đích ứng dụng:	Error! Bookmark not defined.
1.4.2. Mục đích học tập:	Error! Bookmark not defined.
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	11
2.1. Thị giác máy tính (Computer Vision)	11
2.1.1. OpenCV và xử lý video real-time.....	Error! Bookmark not defined.
2.1.2. Biểu diễn ảnh số và không gian màu	11
2.2. MediaPipe	12
2.2.1. Cơ chế tracking và smoothing	19
2.3. Thị giác máy tính với OpenCV	Error! Bookmark not defined.
2.4. Cấu trúc dữ liệu thời gian: deque và Counter	20
2.5. Notebook huấn luyện và quy trình MLOps tối giản.....	Error! Bookmark not defined.
2.6. Công nghệ Text-to-Speech	20
CHƯƠNG 3. TRIỂN KHAI ĐỀ TÀI	22
3.1. Mô hình hệ thống	22
3.2. Đầu vào và đầu ra	25
3.2.1. Đầu vào.....	25
3.2.2. Đầu ra.....	25
3.3. Quy trình xử lý	26
CHƯƠNG 4. KẾT LUẬN	29
4.1. Kết quả đạt được.....	29
4.2. Hạn chế và khó khăn	30
4.3. Hướng phát triển	30
CHƯƠNG 5. TÀI LIỆU THAM KHẢO	34
PHỤ LỤC.....	35
5.1. Phụ lục A:	Error! Bookmark not defined.

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án môn học, chúng em đã nhận được sự hướng dẫn tận tình và hỗ trợ quý báu từ cô. Chúng em xin gửi lời cảm ơn chân thành đến cô **Nguyễn Thị Mỹ Hạnh**, người luôn tận tâm chỉ dẫn, giúp chúng em định hướng phương pháp nghiên cứu, giải đáp những thắc mắc và tạo điều kiện thuận lợi để chúng em hoàn thành tốt đồ án này.

Chúng em cũng xin cảm ơn các Thầy/Cô trong bộ môn **Thị Giác Máy Tính** đã truyền đạt cho chúng em những kiến thức nền tảng quan trọng trong suốt quá trình học tập, là cơ sở để chúng em vận dụng vào việc thực hiện đồ án. Mặc dù đã cố gắng hết sức, bài làm vẫn khó tránh khỏi những thiếu sót do hạn chế về thời gian và kiến thức. Chúng em rất mong nhận được những góp ý từ Cô để chúng em có thể hoàn thiện hơn trong những lần thực hiện đồ án sau.

Chúng em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIẢNG VIÊN

DANH MỤC HÌNH

Hình 1.1. Quy trình nhận diện ngôn ngữ ký hiệu	Error! Bookmark not defined.
Hình 2.1. Kiến trúc MediaPipe Hands pipeline	Error! Bookmark not defined.
Hình 2.2. Hand landmarks [3]	Error! Bookmark not defined.
Hình 3.1. Kiến trúc tổng thể hệ thống	Error! Bookmark not defined.
Hình 3.2. Sơ đồ qui trình xử lý chi tiết	Error! Bookmark not defined.

DANH MỤC BẢNG

Bảng 2.1. So sánh và giải thích các tham số cấu hình.....	18
Bảng 3.1. Qui trình xử lý một khung hình.....	Error! Bookmark not defined.
Bảng 6.1. Bảng phân công công việc	Error! Bookmark not defined.

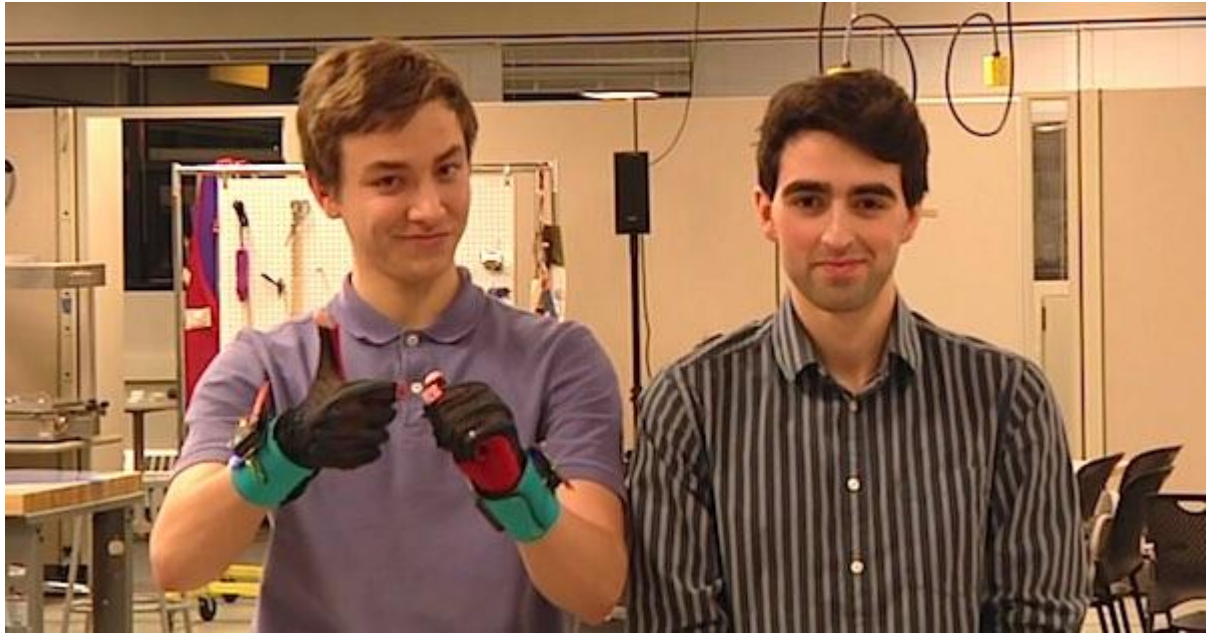
CHƯƠNG 1: TỔNG QUAN

1.1. Đặt vấn đề

Theo thống kê của Tổ chức Y tế Thế giới (WHO), hiện có khoảng 466 triệu người trên toàn cầu bị khiếm thính ở mức độ khác nhau, chiếm khoảng 5% dân số thế giới. Tại Việt Nam, con số này lên tới hơn 8 triệu người. Cộng đồng người khiếm thính gặp rất nhiều khó khăn trong giao tiếp hàng ngày, đặc biệt khi tương tác với những người không biết ngôn ngữ ký hiệu. [1]

Ngôn ngữ ký hiệu (Sign Language) là phương tiện giao tiếp chủ yếu của cộng đồng người điếc và khiếm thính. Tuy nhiên, không phải ai cũng có khả năng học và sử dụng thành thạo ngôn ngữ này. Điều này tạo ra rào cản lớn trong việc hòa nhập xã hội của người khiếm thính, khiến họ gặp khó khăn trong việc giao tiếp với người thân, bạn bè, đồng nghiệp và các dịch vụ công cộng.

Với sự phát triển mạnh mẽ của công nghệ thị giác máy tính (Computer Vision) và trí tuệ nhân tạo (AI) trong những năm gần đây, việc ứng dụng công nghệ để giải quyết vấn đề này trở nên khả thi hơn bao giờ hết. Các công nghệ như MediaPipe Hands của Google cho phép phát hiện và theo dõi bàn tay với độ chính xác cao trong thời gian thực. Kết hợp với các thuật toán nhận dạng cử chỉ và công nghệ chuyển văn bản thành giọng nói (Text-to-Speech), chúng ta có thể xây dựng một hệ thống hỗ trợ giao tiếp tự động. [2]



Hình 1: Ý tưởng từ các học sinh MIT

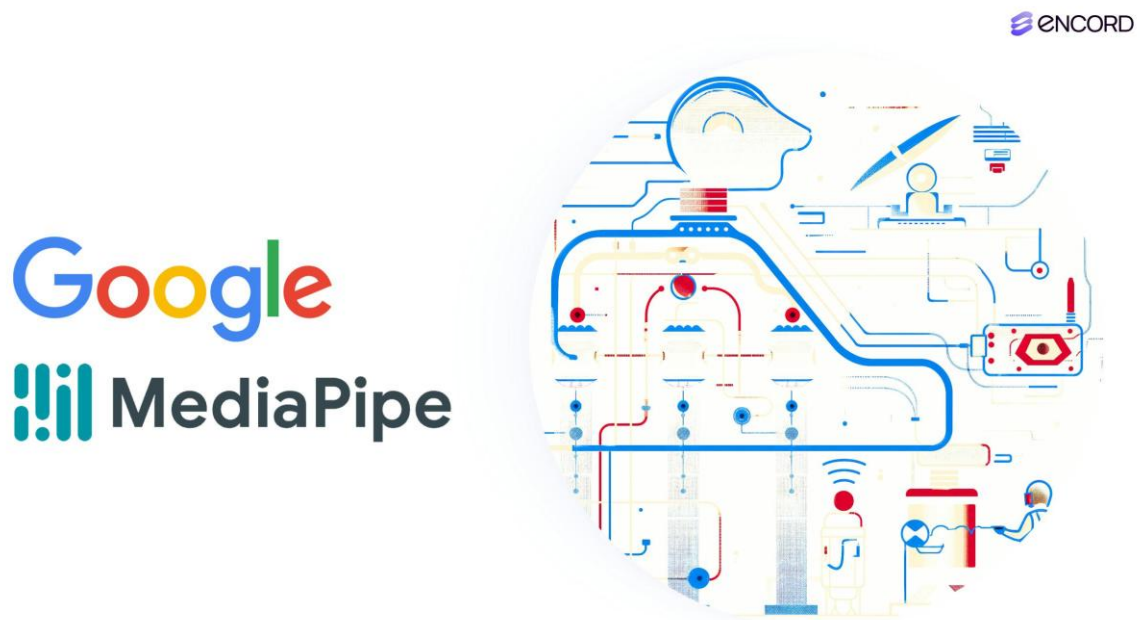
Hệ thống như vậy sẽ giúp người khiếm thính có thể giao tiếp với người không biết ngôn ngữ ký hiệu thông qua việc chuyển đổi cử chỉ tay thành giọng nói. Đồng thời, hệ thống cũng có thể được sử dụng như một công cụ học tập và thực hành ngôn ngữ ký hiệu cho những người muốn học. Đây chính là động lực để nhóm chúng em thực hiện đề tài "Nhận diện ngôn ngữ ký hiệu và chuyển đổi thành giọng nói" nhằm ứng dụng kiến thức Computer Vision đã học để giải quyết một vấn đề thực tiễn có ý nghĩa xã hội cao.[3]

1.2. Mục tiêu đề tài

Đề tài “Nhận diện cử chỉ tay thời gian thực và chuyển đổi thành giọng nói” được thực hiện nhằm hướng tới các mục tiêu chính sau đây:

Mục tiêu tổng quát: Xây dựng thành công một hệ thống thị giác máy tính có khả năng phát hiện, phân tích và nhận diện các cử chỉ tay của người dùng thông qua camera theo thời gian thực (real-time). Từ đó, chuyển đổi các tín hiệu cử chỉ thành văn bản và giọng nói tự nhiên, tạo ra một công cụ hỗ trợ giao tiếp hiệu quả cho cộng đồng người khiếm thính.

Mục tiêu cụ thể: Về mặt ứng dụng và ý nghĩa xã hội: Phát triển giải pháp công nghệ nhằm xóa bỏ rào cản ngôn ngữ, giúp người khiếm thính hòa nhập cộng đồng và tương tác thuận lợi hơn với những người không biết ngôn ngữ ký hiệu. Xây dựng phương thức tương tác người - máy tự nhiên (Human-Computer Interaction) không cần sử dụng các thiết bị vật lý như chuột hay bàn phím, minh họa cho tính ứng dụng thực tiễn cao của trí tuệ nhân tạo (AI) trong đời sống. Về mặt kỹ thuật và công nghệ: Ứng dụng hiệu quả các công nghệ tiên tiến như MediaPipe Hands (của Google) để phát hiện bàn tay với độ chính xác cao và tốc độ xử lý nhanh.[4]

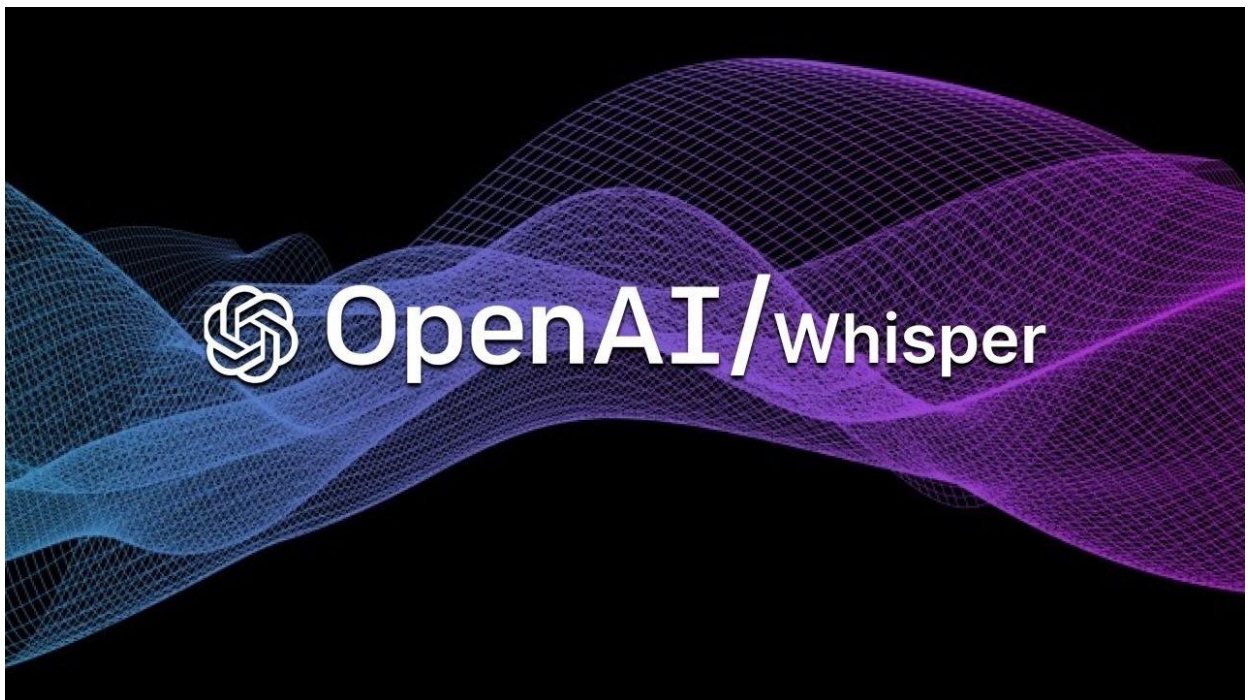


Hình 2 Thư Viện Mediapipe của Google

Tích hợp công nghệ chuyển đổi văn bản thành giọng nói (Text-to-Speech - cụ thể là OpenAI TTS API) để tạo ra đầu ra âm thanh tự nhiên, dễ nghe. Đảm bảo hệ thống hoạt

động ổn định, có khả năng mở rộng để nhận diện thêm nhiều cử chỉ phức tạp, biểu cảm khuôn mặt hoặc cử chỉ cơ thể trong tương lai.

Về mặt học tập và nghiên cứu: Củng cố và áp dụng các kiến thức chuyên ngành Computer Vision (Thị giác máy tính) đã học, bao gồm: xử lý ảnh, phát hiện đối tượng, nhận dạng mẫu và trích xuất đặc trưng. Rèn luyện kỹ năng xây dựng một quy trình học máy (Machine Learning pipeline) hoàn chỉnh: từ thu thập, xử lý dữ liệu, huấn luyện mô hình đến đánh giá và tối ưu hóa thuật toán. Nâng cao kỹ năng lập trình và tư duy giải quyết bài toán thực tế thông qua việc triển khai một sản phẩm công nghệ hoàn thiện.[5]



Hình 3: Mô Hình Whisper của OpenAI

1.3. Lý do chọn đề tài

Mục đích chính của đề tài là xây dựng một hệ thống nhận diện ngôn ngữ ký hiệu thời gian thực và chuyển đổi thành giọng nói. Hệ thống này nhằm hỗ trợ người khiếm thính giao tiếp với những người không biết ngôn ngữ ký hiệu thông qua việc tự động chuyển đổi cử chỉ tay thành giọng nói.

Về mặt kỹ thuật, đề tài hướng tới việc xây dựng một hệ thống có khả năng phát hiện bàn tay và trích xuất các điểm landmark với độ chính xác cao, nhận dạng các cử chỉ tay phổ biến trong ngôn ngữ ký hiệu, và chuyển đổi các cử chỉ được nhận dạng thành văn

bản rồi phát ra giọng nói tự nhiên. Hệ thống cần hoạt động trong thời gian thực với tốc độ xử lý đủ nhanh để đảm bảo trải nghiệm người dùng mượt mà.

Về mặt ứng dụng, đề tài nhằm tạo ra một công cụ thực tế có thể được sử dụng trong các tình huống giao tiếp hàng ngày. Hệ thống cần có giao diện trực quan, dễ sử dụng và có khả năng hoạt động ổn định trên các thiết bị phần cứng thông thường mà không yêu cầu cấu hình cao.

Ngoài ra, đề tài cũng nhằm mục đích học tập và nghiên cứu. Qua quá trình thực hiện, chúng em mong muốn hiểu sâu hơn về các kỹ thuật thị giác máy tính, học máy và xử lý tín hiệu. Đồng thời, chúng em cũng hy vọng đề tài này có thể trở thành tài liệu tham khảo hữu ích cho các sinh viên và nhà nghiên cứu khác quan tâm đến lĩnh vực này.[6]

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Thị giác máy tính (Computer Vision)

OpenCV (Open Source Computer Vision Library) là thư viện mã nguồn mở chuyên dụng cho xử lý ảnh và thị giác máy tính, được phát triển bởi Intel từ năm 1999. OpenCV cung cấp hơn 2500 thuật toán tối ưu hóa cho các tác vụ như phát hiện đối tượng, nhận dạng khuôn mặt, theo dõi chuyển động và xử lý video thời gian thực.

Trong đề tài này, OpenCV đóng vai trò nền tảng để thu nhận và xử lý luồng video từ webcam. Hệ thống sử dụng module `cv2.VideoCapture` để truy cập camera, cho phép đọc từng khung hình (frame) với tốc độ cao. Mỗi khung hình được biểu diễn dưới dạng ma trận số có kích thước (height, width, channels), trong đó channels thường là 3 cho ảnh màu RGB hoặc BGR.

Các chức năng chính của OpenCV trong hệ thống bao gồm thu nhận video từ camera với độ phân giải có thể cấu hình (mặc định 640x480 pixels), chuyển đổi không gian màu từ BGR sang RGB để tương thích với MediaPipe, vẽ các thành phần giao diện như bounding box, landmark points và text overlay, và hiển thị kết quả real-time thông qua hàm `cv2.imshow`. [7]

Biểu diễn ảnh số và không gian màu: Ảnh số là một ma trận hai chiều (đối với ảnh xám) hoặc ba chiều (đối với ảnh màu) chứa các giá trị pixel. Mỗi pixel biểu diễn cường độ sáng hoặc màu sắc tại một vị trí cụ thể trong ảnh. Không gian màu BGR (Blue-Green-Red) là định dạng mặc định của OpenCV, trong đó mỗi pixel được biểu diễn bởi ba giá trị từ 0-255 tương ứng với cường độ của kênh xanh lam, xanh lục và đỏ. MediaPipe yêu cầu ảnh đầu vào ở định dạng RGB (Red-Green-Blue), vì vậy hệ thống phải thực hiện chuyển đổi bằng hàm `cv2.cvtColor`.

Công thức chuyển đổi cơ bản:

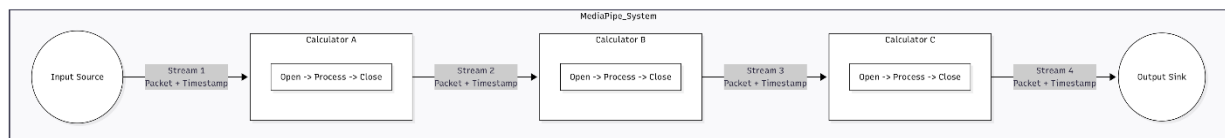
$$\text{RGB} = (\text{B}, \text{G}, \text{R}) \rightarrow (\text{R}, \text{G}, \text{B})$$

Việc chuẩn hóa giá trị pixel về khoảng $[0, 1]$ cũng được thực hiện trong một số trường hợp để tăng hiệu quả xử lý của mô hình học sâu.

2.2 Thư Viện MediaPipe

MediaPipe là một thư viện (framework) mã nguồn mở do Google phát triển, được thiết kế để xây dựng các pipeline xử lý dữ liệu đa phương tiện theo thời gian thực, đặc biệt là dữ liệu hình ảnh và video. Điểm mạnh của MediaPipe nằm ở khả năng cung cấp các mô hình thị giác máy tính (computer vision) tối ưu hóa, có thể chạy hiệu quả trên nhiều nền tảng như máy tính, thiết bị di động và ứng dụng web. Với cấu trúc linh hoạt và hiệu năng cao, MediaPipe cho phép các nhà phát triển dễ dàng tích hợp các chức năng nhận diện, theo dõi và phân tích dữ liệu hình ảnh trong các bài toán như nhận dạng hành động, theo dõi cử chỉ tay, phân tích tư thế cơ thể hay tương tác người-máy. [8]

Kiến trúc MediaPipe hoạt động dựa trên mô hình đồ thị xử lý (Dataflow Graph), trong đó các thành phần xử lý được tổ chức theo dạng một pipeline linh hoạt và có thể mở rộng. Trong mô hình này, dữ liệu không được xử lý tuyến tính mà được truyền qua nhiều khối xử lý độc lập gọi là **Calculator**, thông qua các luồng dữ liệu (streams). Toàn bộ cấu trúc này được mô tả trong các file cấu hình Protobuf (.pbtxt), cho phép nhà phát triển định nghĩa rõ ràng các bước xử lý, cách liên kết giữa các thành phần và quy tắc đồng bộ hóa dữ liệu.



Hình 4 Kiến Trúc Mediapipe

Cốt lõi của kiến trúc MediaPipe bao gồm 3 thành phần chính: **Graph**, **Packet** và **Calculator**:

Graph: là lớp điều phối trung tâm, mô tả mối quan hệ và luồng xử lý giữa các **Calculator**. Graph đảm nhiệm lập lịch, đồng bộ thời gian và điều hướng dữ liệu trong hệ thống. Đặc biệt, cách tổ chức đồ thị giúp MediaPipe dễ dàng cho phép mở rộng, tái sử dụng các module và tối ưu hiệu năng đa luồng

Packet: là đơn vị dữ liệu cơ bản được truyền trong Graph. Mỗi Packet chứa dữ liệu thực tế (ảnh, tensor, kết quả nhận dạng...) kèm theo **Timestamp**. Timestamp đóng vai trò then chốt trong việc đồng bộ hóa các luồng, bảo đảm các Calculator chỉ xử lý dữ

liệu khi tất cả đầu vào liên quan có cùng thời điểm. Điều này giúp MediaPipe đạt độ chính xác cao và hạn chế sai lệch dữ liệu trong môi trường thời gian thực

Calculator: là các node xử lý thực hiện những tác vụ cụ thể như tiền xử lý hình ảnh, chạy mô hình học sâu hoặc hậu xử lý dữ liệu. Mỗi calculator có vòng đời gồm ba giai đoạn chính: **Open()** – khởi tạo tài nguyên và cấu hình, **Process()** – xử lý Packet đầu vào và **Close()** – thu hồi tài nguyên.

Kiến trúc dạng mô-đun này không chỉ giúp MediaPipe dễ tùy biến mà còn tăng khả năng tái sử dụng và tối ưu hóa. Trong quá trình vận hành các Calculator không chạy trực tiếp theo lệnh người dùng mà được **Graph** điều phối ngầm, khi dữ liệu đầu vào đến và các Timestamp giữa các luồng đã đồng bộ thì Graph sẽ tự động kích hoạt hàm **Process()** của Calculator tương ứng, nhờ đó toàn bộ pipeline hoạt động liên tục và song song, đảm bảo dữ liệu được xử lý đúng thời điểm trong môi trường thời gian thực [9]

Quy trình vận hành tổng thể:

Khởi tạo Graph: hệ thống đọc file cấu hình .pbtxt để biết các Calculator nào sẽ tham gia và cách chúng kết nối

- Nhận dữ liệu đầu vào: dữ liệu (ảnh, video, tín hiệu) được đóng gói thành Packet kèm Timestamp
- Truyền Packet qua Graph: Graph điều phối Packet đến đúng Calculator theo sơ đồ
- Calculator xử lý: mỗi Calculator nhận Packet, chạy Process(), rồi phát Packet mới ra
- Đồng bộ thời gian: Graph chỉ cho phép Calculator chạy khi tất cả Packet đầu vào có cùng Timestamp
- Đầu ra cuối cùng: kết quả (landmark bàn tay, khuôn mặt, pose,...) được phát ra từ Graph cho ứng dụng sử dụng

Nhờ kiến trúc dựa trên đồ thị cùng cơ chế đồng bộ hóa theo Timestamp, MediaPipe đạt được khả năng xử lý dữ liệu thời gian thực với độ trễ thấp, phù hợp cho các ứng dụng đòi hỏi hiệu năng cao.

Và khi nhắc tới đề tài nhận diện cử chỉ tay theo thời gian thực thì trong các mô hình phổ biến của Mediapipe người ta thường sử dụng nhiều nhất 3 mô hình: **MediaPipe Hands**, **MediaPipe Pose**, **MediaPipe Holistic**:

MediaPipe Pose là mô hình theo dõi tư thế người (human pose estimation) có khả năng phát hiện và trích xuất các điểm mốc (landmarks) của cơ thể trong ảnh hoặc video. Mô hình nhận diện khoảng 33 điểm trên cơ thể, giúp phân tích cấu trúc xương, chuyển động và tư thế một cách chính xác. Pose thường được dùng trong các lĩnh vực như thể thao, chăm sóc sức khỏe, theo dõi bài tập hoặc phân tích hành vi.

Ưu điểm:

- Tối ưu cho phân tích chuyển động, thể thao, fitness, phát hiện té ngã
- Có thể bật segmentation để tách người khỏi nền
- Có thể chạy realtime trên webcam
- Hiệu năng tốt, có thể chọn mức độ phức tạp (0-2) để cân bằng tốc độ và độ chính xác

Nhược điểm:

- Chỉ tập trung vào cơ thể, không cso khuôn mặt chi tiết hay bàn tay
- Multi-person chưa mạnh bằng các framework khác (thường chỉ người nổi bật nhất).
- Nếu cần thêm thông tin về khuôn mặt hoặc bàn tay thì phải kết hợp thêm mô hình khác

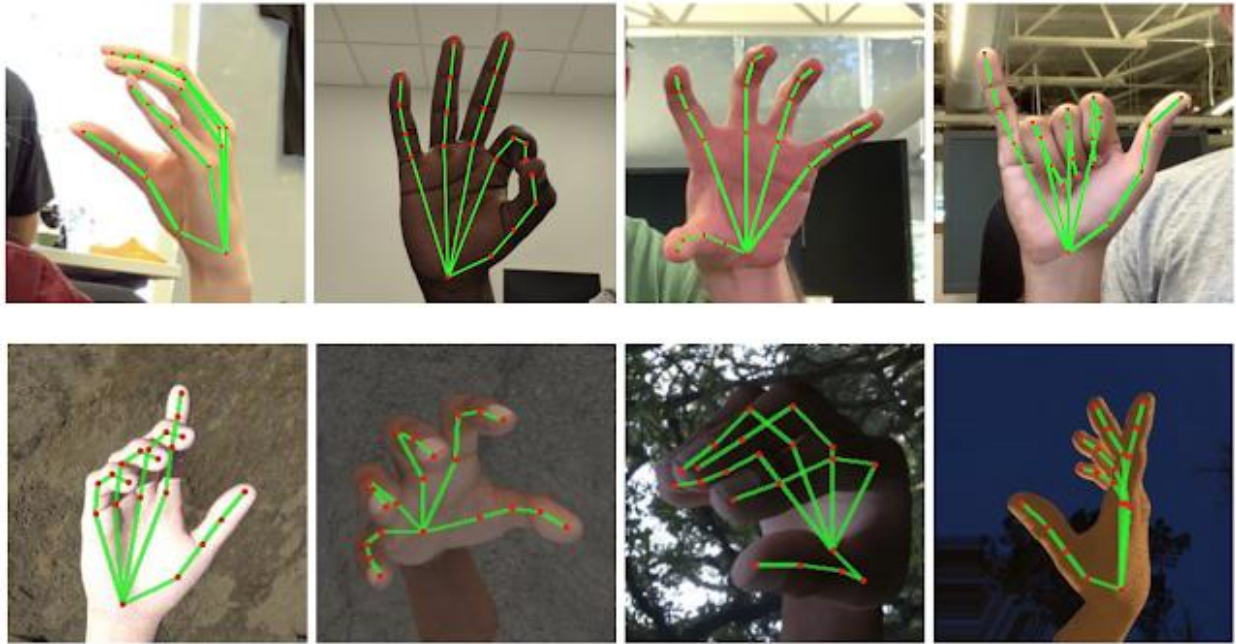
MediaPipe Hands là mô hình nhận diện và theo dõi bàn tay theo thời gian thực. Nó phát hiện vùng bàn tay, sau đó dự đoán 21 điểm mốc ngón tay cho mỗi bàn tay với độ chính xác cao. Nhờ mô hình này, các hệ thống có thể hiểu và phân tích cử chỉ tay (hand gestures), hỗ trợ các ứng dụng như ký hiệu ngôn ngữ, điều khiển không chạm (touchless control) hoặc AR/VR. [10]

Ưu điểm:

- Nhẹ, tối ưu cho nhận diện cử chỉ tay, điều khiển tay, ngôn ngữ ký hiệu
- Độ chính xác cao với bàn tay, có phân loại trái/phải
- Có thể chạy realtime trên webcam

Nhược điểm:

- Chỉ tập trung vào bàn tay, không nhận diện cơ thể hoặc khuôn mặt
- Nếu ảnh bị lật (mirror), cần xử lý thêm để tránh nhầm trái/phải
- Độ chính xác phụ thuộc vào ánh sáng và độ rõ của bàn tay



Hình 5 Hình Ảnh Ứng Dụng Mediapipe Hands

MediaPipe Holistic là mô hình tổng hợp, kết hợp đồng thời ba bộ nhận diện: Face Mesh, Pose và Hands. Holistic cung cấp một bản đồ toàn thân (full-body landmarks) gồm hơn 500 điểm mốc, giúp phân tích chuyển động toàn diện của khuôn mặt, cơ thể và bàn tay trong cùng một pipeline. Mô hình này đặc biệt phù hợp cho các ứng dụng yêu cầu hiểu biết tổng thể về con người, như phân tích tương tác người–máy, mô phỏng nhân vật ảo hoặc nhận diện hành động nâng cao.

Ưu điểm:

- Toàn diện: một lần chạy có đủ pose, face, hand
- Tiện lợi cho ứng dụng AR/VR, avatar, phân tích hành vi phức tạp
- Có thể chạy realtime trên webcam

- Có refine_face_landmarks để tăng độ chính xác vùng mắt/môi/iris => hữu ích cho gaze tracking

Nhược điểm:

- Nặng hơn, tốn CPU/GPU hơn => Có thể giảm FPS trên thiết bị yếu
- Nếu chỉ cần Pose hoặc Hand thì dung riêng sẽ nhẹ và nhanh hơn
- Tốc độ xử lý chậm hơn so với Pose và Hand riêng lẻ

Mô hình	Số điểm mốc	Tốc độ	Độ chính xác	Trường hợp dùng
Hands	21 (1 bàn tay)	Rất nhanh	Rất cao	Ký hiệu ngôn ngữ, điều khiển không chạm, AR/VR
Pose	33	Nhanh	Cao	Nguyên cứu lĩnh vực thể thao, sức khỏe; theo dõi và chăm sóc hành vi
Holistic	543	Trung bình	Rất cao	Tương tác robot, mô phỏng nhân vật, nhận diện hành động nâng cao

Bảng 1 Mô Tả 3 Mô Hình Mediapipe

So sánh và giải thích các tham số cấu hình:

Tham số	Hands	Pose	Holistic	Ý nghĩa
static_image_mode	X	X	X	Quyết định xử lý ảnh tĩnh hay video stream • False (mặc định): coi như video, dung tracking để tiết kiệm nguyên liệu • True: xử lý từng ảnh độc lập, luôn chạy detection
max_num_hands	X			Số lượng bàn tay tối đa mô hình cố gắng phát hiện.

				Mặc định = 2
model_complexity	X	X	X	Độ chi tiết của mô hình: • 0: Nhanh, độ chính xác thấp • 1(mặc định): cân bằng • 2: Chính xác nhất, chậm hơn
min_detection_confidence	X	X	X	Ngưỡng tin cậy để xác nhận phát hiện đối tượng Giá trị 0.0-1.0. Mặc định là 0.5
min_tracking_confidence	X	X	X	Ngưỡng tin cậy để theo dõi landmark thành công. Nếu thấp hơn ngưỡng thì bỏ tracking, chạy lại detection. Giá trị 0.0-1.0. Mặc định là 0.5 Không dùng khi static_image_mode = True
smooth_landmarks		X	X	Làm mượt chuyển động landmarks qua nhiều frame để giảm rung giật Mặc định là True
enable_segmentation		X	X	Bật/tắt mặt nạ phân đoạn người với nền
smooth_segmentation		X	X	Làm mượt segmentation mask qua nhiều frame để giảm nhấp nháy Mặc định là True

refine_face_landmarks			X	Tinh chỉnh thêm landmarks khuôn mặt (mắt, môi, iris)
-----------------------	--	--	---	--

Bảng 2 So sánh và giải thích các tham số cấu hình

MediaPipe Hands là một framework phát hiện và theo dõi bàn tay thời gian thực được phát triển bởi Google Research. Hệ thống sử dụng kiến trúc pipeline hai giai đoạn gồm Palm Detection Model (BlazePalm) và Hand Landmark Model. [11]

Giai đoạn đầu tiên sử dụng BlazePalm, một mô hình CNN nhẹ dựa trên kiến trúc MobileNetV2 và biến thể của BlazeFace, để phát hiện vùng lòng bàn tay trong toàn bộ khung hình. Việc lựa chọn phát hiện “lòng bàn tay” thay vì “bàn tay đầy đủ” xuất phát từ lý do kỹ thuật: lòng bàn tay có hình dạng ổn định hơn, ít biến đổi khi xoay nhiều góc độ, từ đó giúp giảm số lượng anchor, tăng tốc độ xử lý và phù hợp với thiết bị di động. BlazePalm được thiết kế để xử lý các tư thế tay đa dạng với tốc độ cao trên CPU. Mô hình này sử dụng kỹ thuật Single Shot Detection (SSD) để dự đoán bounding box định hướng (rotated bounding box), góc xoay (rotation) và confidence score. Kết quả đầu ra của Palm Detector chính là đầu vào trực tiếp cho giai đoạn Landmark. [12]

Giai đoạn thứ hai áp dụng Hand Landmark Model lên vùng bàn tay đã được phát hiện để xác định chính xác vị trí 21 điểm landmark. Mô hình này sử dụng kiến trúc regression network với các lớp convolution và fully-connected để dự đoán tọa độ 3D (x, y, z) của mỗi landmark, trong đó x và y được chuẩn hóa về khoảng [0, 1] theo kích thước ảnh, còn z biểu diễn độ sâu tương đối so với cổ tay. [13]

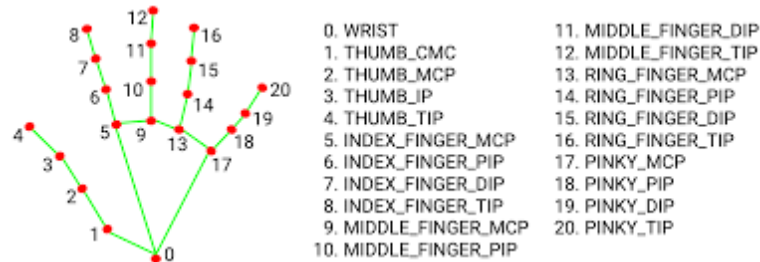
MediaPipe Hands trả về 21 điểm landmark trên bàn tay, được đánh số từ 0 đến 20, bao gồm các vị trí cốt yếu như cổ tay, khớp ngón tay và đầu ngón tay. Cấu trúc 21 điểm này được thiết kế để mô tả đầy đủ hình dạng và tư thế của bàn tay.

Cụ thể, các landmark bao gồm:

- Landmark 0: Cổ tay (wrist) - điểm gốc của bàn tay
- Landmark 1-4: Ngón cái (thumb) - từ khớp gốc đến đầu ngón
- Landmark 5-8: Ngón trỏ (index finger) - từ khớp metacarpal đến đầu ngón

- Landmark 9-12: Ngón giữa (middle finger)
- Landmark 13-16: Ngón áp út (ring finger)
- Landmark 17-20: Ngón út (pinky)

Mỗi nhóm 4 điểm của mỗi ngón tay theo thứ tự CMC (carpometacarpal), MCP (metacarpophalangeal), PIP (proximal interphalangeal), DIP (distal interphalangeal) và TIP (đầu ngón).



Hình 6 Chi tiết các Hand landmarks

Độ của mỗi landmark được biểu diễn dưới dạng (x, y, z) trong đó x và y là tọa độ được chuẩn hóa theo chiều rộng và chiều cao của ảnh, z là độ sâu tương đối so với landmark cổ tay, giá trị càng nhỏ càng gần camera.[13]

Cơ chế tracking và smoothing: Hàm `pre_process_landmark` thực hiện hai thao tác nền tảng: dịch toàn bộ tọa độ về gốc tại landmark thứ nhất (cổ tay), sau đó chuẩn hóa vector bằng giá trị tuyệt đối lớn nhất. Đây là cách đảm bảo bất biến tịnh tiến và tỉ lệ, giúp mô hình chỉ tập trung vào hình dạng tương đối của bàn tay. Việc chuẩn hóa bằng max-norm đơn giản nhưng hiệu quả, giữ các thành phần đầu vào trong miền $[-1, 1]$, từ đó hạn chế hiện tượng gradient vanishing ở các tầng đầu MLP. Cuối cùng, danh sách tọa độ 2D được trải phẳng thành vector một chiều, tương thích với đầu vào fully-connected. Tương tự, `pre_process_point_history` đưa điểm lịch sử về gốc, song song chia theo chiều rộng và chiều cao ảnh nhằm bảo toàn tỷ lệ x/y . Việc cố định phần tử đầu tiên làm gốc bảo đảm vector biểu diễn đường đi tương đối của ngón tay, phù hợp với các lớp cử chỉ mang tính động như “xoay chiều kim đồng hồ” hay “di chuyển”. Cặp bước này cho thấy triết lý của dự án: thay vì dùng mô hình phức tạp, hãy đưa nhiều hiểu biết miền (domain knowledge) vào khâu tiền xử lý để mô hình đơn giản vẫn đạt hiệu quả cao.

Phân loại bàn tay trái.phải (Left/Right Classification): MediaPipe Hands còn tích hợp cơ chế phân biệt bàn tay trái và bàn tay phải. Việc phân loại dựa trên vector hướng lòng bàn tay, vị trí ngón cái so với các ngón khác và hình học bàn tay theo hệ tọa độ camera. Tính năng này đặc biệt quan trọng trong các ứng dụng điều khiển bằng hai tay, nhận diện ký hiệu ngôn ngữ (sign language) hoặc các hệ thống AR/VR cần phân biệt chính xác bàn tay trái và phải.

Cấu trúc dữ liệu thời gian: deque và Counter: collections.deque là cấu trúc hàng đợi hai đầu tối ưu cho thao tác thêm/xóa ở đầu mút, được dùng trong app.py với maxlen=16 cho cả point_history và finger_gesture_history. Điều này tạo ra cửa sổ trượt cố định, nơi dữ liệu cũ tự động bị loại bỏ khi thêm phần tử mới, phù hợp cho phân tích chuỗi thời gian ngắn. Khi point_history chưa đủ độ dài, PointHistoryClassifier sẽ bỏ qua suy luận để tránh kết quả thiếu thông tin. Lịch sử nhận finger_gesture được tổng hợp bằng Counter.most_common, tương đương với bộ lọc đa số nhằm triệt dao động nhận. Về mặt lý thuyết, đây là dạng smoothing phi tham số, tận dụng tính chất rằng lỗi suy luận thường độc lập giữa các khung liên tiếp nên biểu quyết đa số sẽ trả về nhận đúng với xác suất cao hơn. Việc kết hợp hai cơ chế này giúp hệ thống cân bằng giữa phản hồi nhanh và tính ổn định: người dùng chỉ cần giữ cử chỉ trong khoảng nửa giây để nhận hội tụ, tránh nhảy liên tục gây khó khăn cho việc đọc và ghi dữ liệu. [14]

1.4. Công nghệ Text-to-Speech

Text-to-Speech (TTS) là công nghệ chuyển đổi văn bản thành giọng nói. Trong đề tài này, chúng em sử dụng OpenAI TTS API, một dịch vụ TTS hiện đại sử dụng mô hình học sâu để tạo ra giọng nói tự nhiên và dễ nghe.

OpenAI TTS API cung cấp nhiều mô hình và giọng nói khác nhau. Mô hình tts-1 được sử dụng trong đề tài này là mô hình cơ bản nhưng vẫn đảm bảo chất lượng giọng nói tốt và tốc độ xử lý nhanh. API hỗ trợ nhiều giọng nói khác nhau như alloy, echo, fable, onyx, nova và shimmer, mỗi giọng nói có đặc điểm riêng về âm sắc và phong cách.

Quá trình chuyển đổi văn bản thành giọng nói bắt đầu khi hệ thống nhận dạng được một cử chỉ và chuyển đổi nó thành văn bản. Văn bản này được gửi đến OpenAI TTS API

thông qua một yêu cầu HTTP. API xử lý văn bản và trả về một file audio dưới dạng MP3. File audio này sau đó được phát lại bằng thư viện Pygame để người dùng có thể nghe được giọng nói.

Một tính năng quan trọng của hệ thống là Speech Buffer, một cơ chế tích lũy các ký tự được nhận dạng thành từ và câu hoàn chỉnh trước khi phát giọng nói. Điều này giúp hệ thống có thể tạo ra các câu nói có ý nghĩa thay vì chỉ phát từng ký tự riêng lẻ. Người dùng có thể điều khiển quá trình này thông qua các phím tắt như thêm khoảng trắng, xóa ký tự cuối, hoặc phát giọng nói ngay lập tức.

Việc sử dụng API bên ngoài như OpenAI TTS có ưu điểm là chất lượng giọng nói cao và không cần phải xây dựng và huấn luyện mô hình TTS riêng. Tuy nhiên, nó cũng có nhược điểm là phụ thuộc vào kết nối internet và có thể phát sinh chi phí sử dụng. Trong tương lai, hệ thống có thể được cải tiến để hỗ trợ các giải pháp TTS offline hoặc cho phép người dùng lựa chọn giữa các phương án khác nhau. [15]

CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI

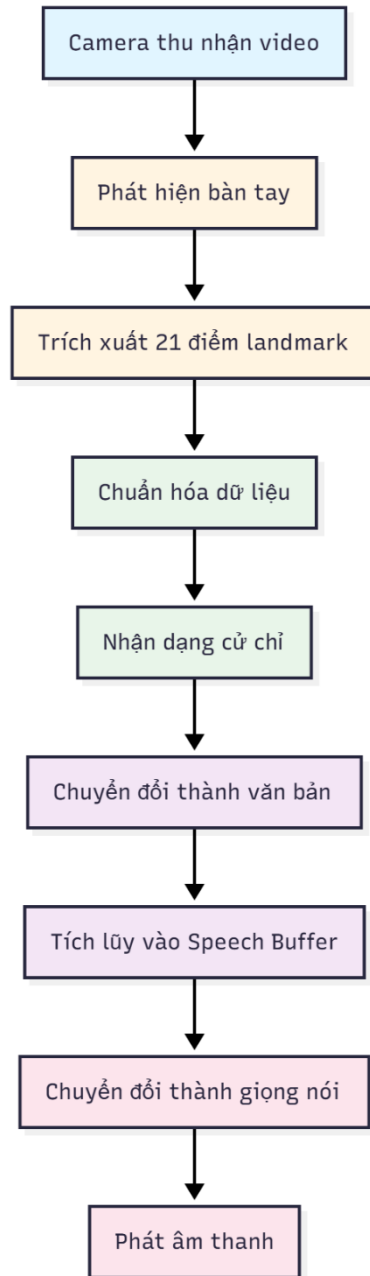
3.1 Mô hình hệ thống

Hệ thống nhận diện ngôn ngữ ký hiệu được thiết kế theo kiến trúc module hóa, cho phép dễ dàng bảo trì, mở rộng và thay thế các thành phần. Kiến trúc tổng thể của hệ thống bao gồm ba lớp chính: lớp thu nhận và tiền xử lý, lớp suy luận và nhận dạng, và lớp giao tiếp và phản hồi.

Lớp thu nhận và tiền xử lý chịu trách nhiệm thu nhận video từ camera, chuyển đổi không gian màu, và phát hiện bàn tay cùng các điểm landmark. Lớp này sử dụng OpenCV để thu nhận video và MediaPipe Hands để phát hiện bàn tay. Dữ liệu sau khi được xử lý sẽ được chuyển đến lớp tiếp theo dưới dạng danh sách các điểm landmark.

Lớp suy luận và nhận dạng nhận dữ liệu landmark từ lớp trước, thực hiện chuẩn hóa dữ liệu, và sử dụng các mô hình học máy để nhận dạng cử chỉ. Lớp này bao gồm hai bộ phân loại độc lập: một cho cử chỉ tĩnh và một cho cử chỉ động. Kết quả nhận dạng được trả về dưới dạng tên cử chỉ và độ tin cậy.

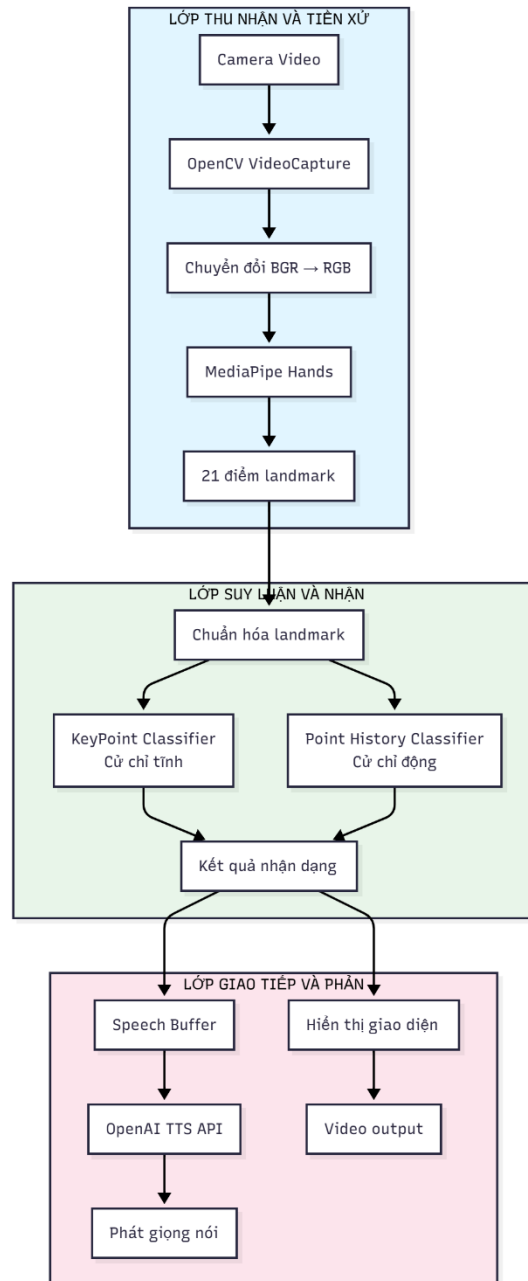
Lớp giao tiếp và phản hồi chịu trách nhiệm hiển thị kết quả lên màn hình, quản lý Speech Buffer, và gửi yêu cầu đến OpenAI TTS API để tạo giọng nói. Lớp này cũng xử lý các tương tác từ người dùng thông qua bàn phím và cập nhật giao diện theo thời gian thực.



Hình 7: Quy trình nhận diện ngôn ngữ ký hiệu

Luồng xử lý chính của hệ thống bắt đầu từ việc đọc một khung hình từ camera. Khung hình này được chuyển đổi từ định dạng BGR sang RGB và được gửi đến MediaPipe Hands để phát hiện bàn tay. Nếu phát hiện được bàn tay, hệ thống sẽ trích xuất 21 điểm landmark và chuẩn hóa chúng. Dữ liệu đã chuẩn hóa được đưa vào các bộ phân loại để nhận dạng cử chỉ. Kết quả nhận dạng được hiển thị lên màn hình và được thêm vào Speech Buffer.

Khi người dùng yêu cầu, hệ thống sẽ gửi nội dung trong Speech Buffer đến OpenAI TTS API để tạo giọng nói.



Hình 8 Kiến trúc tổng thể hệ thống

3.2 Đầu vào và đầu ra

3.2.1 Đầu vào

Đầu vào chính của hệ thống là luồng video từ webcam. Hệ thống đọc từng khung hình từ camera với độ phân giải có thể cấu hình, thường là 640x480 hoặc 1280x720 pixels. Mỗi khung hình là một ma trận ba chiều biểu diễn ảnh màu RGB.

Ngoài đầu vào video, hệ thống còn nhận các tham số cấu hình từ file môi trường. - Các tham số này bao gồm chỉ số camera, độ phân giải, ngưỡng tin cậy cho phát hiện và theo dõi bàn tay, khóa API cho OpenAI TTS, và các tùy chọn hiển thị. Việc sử dụng file cấu hình cho phép người dùng dễ dàng điều chỉnh hệ thống mà không cần sửa đổi mã nguồn.

Hệ thống cũng hỗ trợ tương tác từ người dùng thông qua bàn phím. Người dùng có thể sử dụng các phím tắt để điều khiển hệ thống, chẳng hạn như thêm khoảng trắng, xóa ký tự, phát giọng nói, tạm dừng hoặc tiếp tục xử lý, và thoát khỏi ứng dụng.

3.2.2 Đầu ra

Đầu ra chính của hệ thống là giọng nói được tạo ra từ các cử chỉ được nhận dạng. - Giọng nói này được phát qua loa của máy tính, cho phép người dùng nghe được nội dung tương ứng với các cử chỉ mà họ thực hiện.

Ngoài đầu ra âm thanh, hệ thống còn hiển thị thông tin trực quan trên màn hình. - Giao diện hiển thị bao gồm video từ camera với các điểm landmark được vẽ lên, bounding box xung quanh bàn tay, tên cử chỉ được nhận dạng, độ tin cậy, nội dung trong Speech Buffer, và các thông tin trạng thái khác như FPS và hướng dẫn sử dụng.

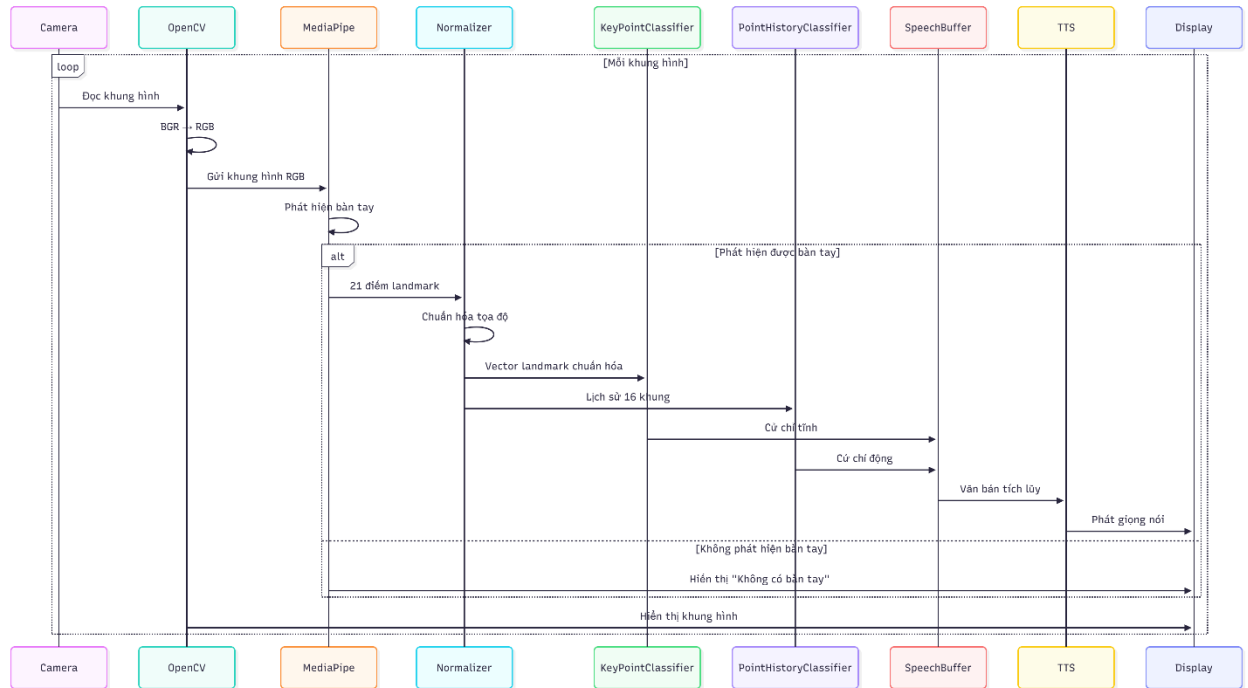
Hệ thống cũng có khả năng ghi lại dữ liệu huấn luyện khi người dùng kích hoạt chế độ thu thập dữ liệu. Dữ liệu này được lưu trữ dưới dạng file CSV và có thể được sử dụng để cải thiện hoặc mở rộng các mô hình nhận dạng trong tương lai.

3.3 Quy trình xử lý

Quy trình xử lý của hệ thống được thực hiện trong một vòng lặp chính, mỗi lần lặp xử lý một khung hình từ camera. Vòng lặp này tiếp tục cho đến khi người dùng yêu cầu thoát hoặc có lỗi xảy ra.

Bước	Mô tả	Công nghệ sử dụng
1	Đọc khung hình từ camera	OpenCV VideoCapture
2	Chuyển đổi không gian màu BGR sang RGB	OpenCV cvtColor
3	Phát hiện bàn tay và trích xuất landmark	MediaPipe Hands
4	Chuẩn hóa tọa độ landmark	Thuật toán chuẩn hóa
5	Nhận dạng cử chỉ tĩnh	KeyPoint Classifier (TFLite)
6	Nhận dạng cử chỉ động	Point History Classifier(TFLite)
7	Cập nhật Speech Buffer	Logic xử lý trạng thái
8	Hiển thị kết quả lên màn hình	OpenCV drawing functions
9	Xử lý tương tác người dùng	Keyboard event handling

Bảng 3 Qui trình xử lý một khung hình



Hình 9 Sơ đồ qui trình xử lý chi tiết

Bước đầu tiên trong mỗi vòng lặp là đọc một khung hình từ camera. Khung hình này được kiểm tra để đảm bảo đọc thành công. Nếu không đọc được, hệ thống sẽ báo lỗi và kết thúc vòng lặp.

Sau khi đọc được khung hình, hệ thống chuyển đổi nó từ định dạng BGR sang RGB để tương thích với MediaPipe. Khung hình RGB sau đó được gửi đến MediaPipe Hands để phát hiện bàn tay. MediaPipe trả về thông tin về bàn tay được phát hiện, bao gồm 21 điểm landmark và thông tin về tay trái hoặc tay phải.

Nếu phát hiện được bàn tay, hệ thống sẽ trích xuất các điểm landmark và chuyển đổi chúng từ tọa độ chuẩn hóa sang tọa độ pixel để hiển thị. Đồng thời, hệ thống cũng chuẩn hóa các điểm landmark bằng cách dịch chúng về gốc tại cổ tay và chia cho giá trị lớn nhất để đảm bảo tính bất biến.

Dữ liệu đã chuẩn hóa được đưa vào các bộ phân loại để nhận dạng cử chỉ. Bộ phân loại KeyPoint nhận dạng cử chỉ tĩnh dựa trên vị trí hiện tại của các điểm landmark. Bộ

phân loại Point History nhận dạng cử chỉ động dựa trên lịch sử di chuyển của các điểm landmark trong 16 khung hình gần nhất.

Kết quả nhận dạng được xử lý và cập nhật vào trạng thái của ứng dụng. Nếu cử chỉ được nhận dạng với độ tin cậy cao và được giữ trong một khoảng thời gian đủ lâu, nó sẽ được thêm vào Speech Buffer. Hệ thống cũng có thể tự động phát giọng nói cho cử chỉ được nhận dạng tùy theo cấu hình. [16]

Cuối cùng, hệ thống vẽ các thành phần giao diện lên khung hình, bao gồm các điểm landmark, bounding box, thông tin cử chỉ, và các thành phần khác. Khung hình đã được xử lý được hiển thị trên màn hình, và hệ thống kiểm tra xem có phím nào được nhấn không để xử lý các tương tác từ người dùng.

Quy trình này được lặp lại liên tục với tốc độ cao, thường đạt từ 30 đến 60 khung hình mỗi giây tùy thuộc vào cấu hình phần cứng và độ phức tạp của xử lý. Điều này đảm bảo hệ thống có thể phản hồi nhanh chóng và mượt mà với các cử chỉ của người dùng.

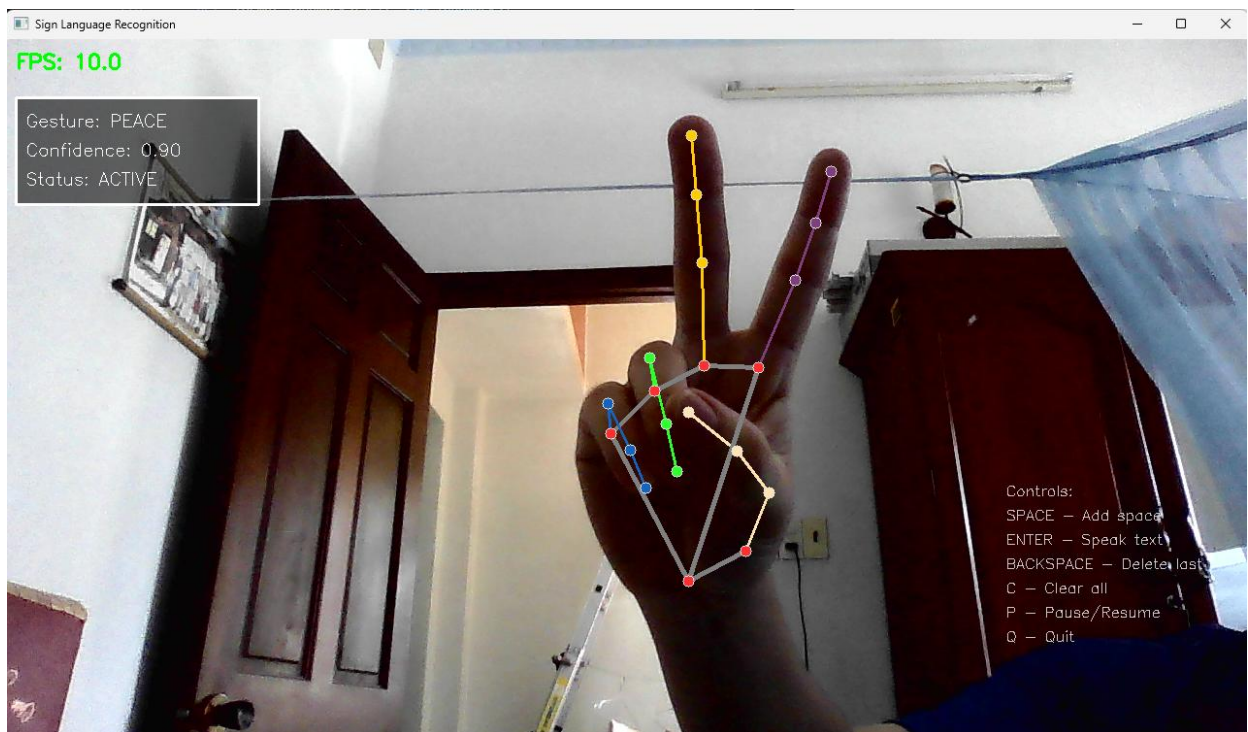
CHƯƠNG 4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Qua quá trình nghiên cứu và triển khai, nhóm chúng em đã xây dựng thành công một hệ thống nhận diện ngôn ngữ ký hiệu và chuyển đổi thành giọng nói. Hệ thống có khả năng phát hiện bàn tay và trích xuất các điểm landmark với độ chính xác cao, nhận dạng các cử chỉ tay phổ biến, và chuyển đổi chúng thành giọng nói tự nhiên.

Về mặt kỹ thuật, hệ thống đã đạt được các mục tiêu ban đầu. Việc sử dụng MediaPipe Hands cho phép phát hiện bàn tay với tốc độ cao và độ chính xác tốt. Các mô hình học máy được huấn luyện và tối ưu hóa bằng TensorFlow Lite giúp hệ thống có thể chạy mượt mà trên CPU thông thường. Tích hợp OpenAI TTS API mang lại chất lượng giọng nói tự nhiên và dễ nghe.

Về mặt ứng dụng, hệ thống đã tạo ra một công cụ thực tế có thể được sử dụng trong các tình huống giao tiếp hàng ngày. Giao diện trực quan và dễ sử dụng giúp người dùng có thể tương tác với hệ thống một cách thuận tiện. Khả năng hoạt động ổn định trên các thiết bị phần cứng thông thường làm tăng tính khả dụng của hệ thống.



Hình 10 Demo Ứng Dụng

Màn hình sẽ hiển thị các landmark của bàn tay cùng với những tùy chọn bàn phím và bắt đầu tính toán thông số và cho ra kết quả: Gesture: PEACE, Confidence: 0.90, Status: Active và sau đó sẽ phát ra giọng đọc “Hello”

4.2 Hạn chế và khó khăn

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Thứ nhất, hệ thống chỉ hỗ trợ nhận dạng một bàn tay tại một thời điểm, điều này có thể gây khó khăn trong một số tình huống giao tiếp thực tế. Thứ hai, hệ thống phụ thuộc vào điều kiện ánh sáng và môi trường để đảm bảo độ chính xác. Trong điều kiện ánh sáng kém hoặc môi trường phức tạp, khả năng phát hiện và nhận dạng có thể bị giảm sút.

Thứ ba, việc sử dụng API bên ngoài cho TTS yêu cầu kết nối internet và có thể phát sinh chi phí. Điều này có thể gây khó khăn cho người dùng ở những nơi không có kết nối internet ổn định hoặc có ngân sách hạn chế. Ngoài ra, hệ thống hiện tại chỉ hỗ trợ một số cử chỉ cơ bản, chưa bao phủ toàn bộ bảng chữ cái ngôn ngữ ký hiệu.

Trong quá trình thực hiện, nhóm chúng em cũng gặp một số khó khăn. Việc thu thập và gán nhãn dữ liệu huấn luyện là một công việc tốn thời gian và đòi hỏi sự cẩn thận. Việc tối ưu hóa hiệu năng để đảm bảo hệ thống chạy mượt mà trên các thiết bị khác nhau cũng là một thách thức. Ngoài ra, việc tích hợp các thành phần khác nhau và đảm bảo chúng hoạt động ổn định cùng nhau đòi hỏi nhiều thời gian kiểm thử và điều chỉnh.

4.3 Hướng phát triển Ứng dụng YOLOv8 cho phân đoạn đối tượng

Bên cạnh phương pháp trích xuất đặc trưng hình học (Landmark-based) sử dụng MediaPipe, nhóm nghiên cứu đề xuất một hướng tiếp cận nâng cao sử dụng mạng nơ-ron tích chập sâu (Deep CNN) để giải quyết các hạn chế về nhiễu nền và điều kiện ánh sáng. Cụ thể, mô hình **YOLOv8 (You Only Look Once - version 8)** với task **Instance Segmentation** được lựa chọn để phân tích. [17]

4.3.1. Tổng quan về YOLOv8 và bài toán Instance Segmentation

YOLOv8 là kiến trúc SOTA (State-of-the-Art) được phát triển bởi Ultralytics, nổi bật với khả năng cân bằng giữa độ chính xác (mAP) và tốc độ thực thi (FPS), yếu tố cốt lõi cho các ứng dụng thời gian thực như nhận diện ngôn ngữ ký hiệu.

Khác với bài toán phát hiện vật thể (Object Detection) chỉ trả về khung bao hình chữ nhật (Bounding Box), **Instance Segmentation** trả về một mặt nạ (Mask) bao quanh chính xác từng pixel của đối tượng. Đối với bài toán nhận diện cử chỉ tay, điều này có ý nghĩa quan trọng:

- **Bounding Box (YOLO thường):** Bao gồm cả vùng nền (background) gây nhiễu nếu tay đặt trước vật thể phức tạp.
- **Segmentation Mask (YOLOv8-seg):** Tách biệt hoàn toàn bàn tay khỏi nền, giúp loại bỏ nhiễu và cung cấp thông tin hình dáng (shape) chi tiết hơn của các ngón tay.

4.3.2 Kiến trúc mạng YOLOv8 (Network Architecture)

YOLOv8 giới thiệu những cải tiến đáng kể so với các phiên bản tiền nhiệm (v5, v7), chuyển sang cơ chế **Anchor-Free** (không sử dụng khung neo định trước) và kiến trúc mới. Cấu trúc tổng thể gồm 3 phần chính:

1. Backbone (Mạng trích xuất đặc trưng): YOLOv8 sử dụng biến thể của CSPDarknet53 (Cross Stage Partial Network). Thay thế khối C3 (của YOLOv5) bằng khối **C2f (Cross Stage Partial with 2 bottlenecks)**.

Cơ chế: Khối C2f giúp dòng gradient lan truyền tốt hơn qua mạng, cho phép trích xuất các đặc trưng ngữ nghĩa mức cao (như hình dạng ngón tay, kết cấu da) và mức thấp (cạnh, góc) hiệu quả hơn mà vẫn giữ nguyên trọng số nhẹ. Lớp **SPPF (Spatial Pyramid Pooling - Fast)** ở cuối backbone giúp mô hình nhận diện tốt các đối tượng ở nhiều tỷ lệ kích thước khác nhau (tay gần hoặc xa camera).

2. Neck (Mạng kết hợp đặc trưng): Sử dụng kiến trúc PANet (Path Aggregation Network) để trộn các đặc trưng từ các tầng khác nhau của Backbone. Điều này đảm bảo rằng thông tin chi tiết về vị trí (từ các lớp thấp) và thông tin ngữ nghĩa (từ các lớp cao) được kết hợp, giúp phát hiện chính xác bàn tay nhỏ hoặc bị che khuất một phần.

3. Head (Phần đầu ra dự đoán): YOLOv8 sử dụng cơ chế Decoupled Head (Đầu ra tách biệt). Thay vì gộp chung, mạng tách luồng xử lý thành các nhánh riêng biệt:

1. **Classification Branch:** Dự đoán lớp đối tượng (ví dụ: tay trái, tay phải, hoặc các ký tự A, B, C...).

2. **Regression Branch:** Dự đoán tọa độ khung bao (Box coordinates).

3. **Segmentation Branch:** Dự đoán mặt nạ phân đoạn (Mask). [17]

4.3.3. Giải thích thuật toán phân đoạn trong YOLOv8

Điểm đột phá của YOLOv8 trong tác vụ Segmentation là cách tạo ra mặt nạ theo thời gian thực, dựa trên cơ chế tương tự YOLACT (You Only Look At CoefficientTs). Thay vì dự đoán một mặt nạ nặng nề cho mỗi pixel, YOLOv8 chia quy trình thành hai luồng song song:

ProtoNet (Prototype Network): Đây là một mạng con fully-convolutional (FCN).

Nhiệm vụ: Tạo ra một tập hợp k mặt nạ nguyên mẫu (Prototype Masks) có độ phân giải cao cho toàn bộ bức ảnh. Các mặt nạ này chưa đại diện cho một đối tượng cụ thể nào mà chứa các đặc trưng không gian chung của ảnh.

Mask Coefficients (Hệ số mặt nạ): Được dự đoán bởi phần Detection Head cùng với Class và Box. Mỗi đối tượng (bàn tay) được phát hiện sẽ có một bộ k hệ số vô hướng (scalar coefficients) tương ứng với k prototype masks.

Cơ chế tổng hợp (Linear Combination): Mặt nạ cuối cùng của bàn tay được tính toán bằng phép tổ hợp tuyến tính giữa các Prototype Masks và Mask Coefficients, sau đó đi qua hàm kích hoạt Sigmoid:

$$M = \sigma(P \times C^T)$$

Trong đó:

- M : Mặt nạ kết quả (Final Mask).
- P : Ma trận các Prototype Masks.
- C : Ma trận các hệ số (Coefficients).
- σ : Hàm Sigmoid để đưa giá trị về khoảng $[0, 1]$ (xác suất pixel thuộc về bàn tay).

Kết luận: Sau bước này, mặt nạ sẽ được cắt (crop) theo Bounding Box để loại bỏ các vùng thừa bên ngoài, tạo ra kết quả phân đoạn chính xác. [18]

4.3.4. Hàm mất mát (Loss Function)

Để huấn luyện mô hình YOLOv8-seg, hàm mất mát tổng hợp được sử dụng bao gồm:

- **VFL Loss (Varifocal Loss):** Dùng cho phân loại (Classification), giúp tập trung vào các mẫu khó.
- **DFL Loss (Distribution Focal Loss) + CIoU Loss:** Dùng cho hồi quy khung bao (Bounding Box), tối ưu hóa độ khớp của khung hình.

- **Binary Cross-Entropy Loss:** Dùng cho phần Segmentation Mask, tính toán sai số giữa mặt nạ dự đoán và mặt nạ thực tế (Ground Truth) trên từng pixel.

4.3.5. Quy trình ứng dụng đề xuất

Quy trình tích hợp YOLOv8 vào hệ thống nhận diện ngôn ngữ ký hiệu được thiết kế như sau:

1. **Input:** Luồng video từ webcam.
2. **Pre-processing:** Resize ảnh về kích thước tiêu chuẩn của mô hình (ví dụ: 640x 640), chuẩn hóa pixel 0-255 về 0-1.
3. **Inference (YOLOv8-seg):** Mô hình trả về danh sách các đối tượng gồm: Bounding Box, Class ID, Confidence Score và Mask.
4. **Post-processing:** Áp dụng ngưỡng tin cậy (Confidence Threshold > 0.5) để lọc bỏ các dự đoán yếu. Sử dụng thuật toán **NMS (Non-Maximum Suppression)** để loại bỏ các khung hình chồng chéo. Trích xuất vùng mặt nạ tay (Hand Mask) để làm đầu vào cho các bước xử lý tiếp theo (như trích xuất đặc trưng hình học nâng cao hoặc đưa vào mạng phân loại cử chỉ).

Việc áp dụng YOLOv8 hứa hẹn khắc phục điểm yếu của các phương pháp truyền thống trong môi trường nhiễu, đồng thời mở ra khả năng nhận diện các cử chỉ phức tạp dựa trên hình dáng (shape-based) thay vì chỉ dựa trên khung xương.

TÀI LIỆU THAM KHẢO

PHỤ LỤC CODE